

Bitácora de Investigación y Diseño Arquitectónico

Proyecto Co-MIL: Fase de Experimentación y Refinamiento

Parte II

Adrián Emiliano Beltrán Fernández

Facultad de Ciencias, UNAM

Abril 2026

Propósito de la Parte II

Este documento constituye la continuación formal del registro técnico del proyecto Co-MIL. Tras haber consolidado el marco teórico y la infraestructura básica en la Parte I, esta sección se enfoca en el desarrollo de herramientas de procesamiento masivo, la transición hacia la arquitectura MIML (Multi-Instance Multi-Label) de ramas independientes y el registro de la Fase 1 de entrenamiento.

1 Optimización de la Ingesta y Visualización

1.1 Redimensionamiento Dinámico y Protección del Backbone

Se identificó un riesgo de colapso en el extractor *MobileNetV2* al intentar procesar parches de alta densidad (ej. 56x56 px) derivados de capturas en alta resolución (12MP). Dado que el backbone exige una entrada fija de 224x224 px, se implementó una solución a nivel de software en el método `__getitem__(self, idx)` del Dataset.

La operación realiza una interpolación bilineal en tiempo de ejecución, permitiendo que la red trabaje con parches pequeños (maximizando la resolución espacial para el cálculo de áreas en cm^2) sin sacrificar la compatibilidad con los pesos pre-entrenados de ImageNet.

1.2 Reprocesador Headless y Reflection Padding

Para escalar el dataset de 268 imágenes sin intervención manual recurrente, se desarrolló el script `reprocesador_dataset.py`. Este motor extrae las coordenadas originales del experto (`user_bbox`), accede a la imagen `.jpg` cruda y recalcula el *Smart Crop*.

Se eliminó el uso de *Zero-Padding* para evitar bordes negros que inducen falsos gradientes. En su lugar, se emplea `ReflectionPad2d`, el cual mantiene la continuidad biológica del tejido en los límites de la fotografía mediante efecto espejo.

2 Evolución hacia la Arquitectura MIML de K-Ramas

2.1 Desacoplamiento de Mapas de Atención

El esquema inicial de atención global fue sustituido por un mecanismo de atención clase-específico. Bajo esta nueva formulación, el modelo no emite un solo peso α_i por parche, sino un tensor de forma $[Batch, K, N]$, donde K representa el número de tejidos detectados dinámicamente en el dataset.

$$\alpha_i^k = \frac{\exp\{w_k^T(\tanh(Vh_i^T) \odot \sigma(Uh_i^T))\}}{\sum_{j=1}^N \exp\{w_k^T(\tanh(Vh_j^T) \odot \sigma(Uh_j^T))\}} \quad (1)$$

Esta arquitectura permite que el modelo genere mapas de calor independientes: un detector especializado en fibrina, otro en granulación y otro en callo. Esto es fundamental para la futura etapa de segmentación débilmente supervisada (WSSS) y cuantificación de áreas.

3 Fase 1: Motor de Entrenamiento y Optimización

3.1 Función de Pérdida y Mitigación de Sesgo

Para manejar la coexistencia de tejidos, se adoptó la Entropía Cruzada Binaria (BCEWithLogitsLoss). Para contrarrestar el desbalanceo natural de las muestras clínicas, se integró el parámetro *pos_weight* (W_c):

$$l_c = -w_c[y_c \cdot \log \sigma(x_c) + (1 - y_c) \cdot \log(1 - \sigma(x_c))] \quad (2)$$

Donde $w_c > 1$ amplifica el castigo de la red cuando falla en detectar tejidos poco frecuentes, como el tejido necrótico o el tendón, forzando la extracción de características más robustas.

4 Trabajo a Futuro

- **Fase 2:** Evaluación del modelo mediante *Hamming Loss* para medir la precisión en la coexistencia de etiquetas simultáneas.
- **Fase 3:** Implementación del Búfer de Repetición para mitigar el olvido catastrófico en el esquema continuo.
- **Fase 4:** Refinamiento mediante *DenseCRF* para la obtención de máscaras de segmentación de alta fidelidad.

5 Implementación del Motor de Entrenamiento (Fase 1)

5.1 Configuración del Script `entrenar_comil.py`

Se ha consolidado el motor de optimización para la arquitectura MIML. El script integra un ciclo de entrenamiento robusto basado en **AdamW** con una tasa de aprendizaje de 10^{-4} y decaimiento de pesos para prevenir el sobreajuste en el conjunto de datos clínicos.

5.2 Estrategia de Congelamiento y Transferencia de Aprendizaje

Una decisión técnica crítica para la viabilidad computacional en dispositivos móviles es la gestión de los gradientes en los distintos bloques de la red. La estrategia se define bajo los siguientes criterios de entrenamiento:

- **Backbone (MobileNetV2):** Los parámetros se mantienen en estado *frozen* (`requires_grad = False`). Se preservan los filtros convolucionales pre-entrenados en ImageNet para actuar como un extractor de características genérico, reduciendo drásticamente el uso de VRAM y acelerando la convergencia.
- **Mecanismo de Atención (Trainable):** Este módulo se optimiza desde cero. Su objetivo es aprender la importancia relativa de cada instancia dentro de la bolsa, ajustando los pesos α_i^k para filtrar el ruido de fondo (sábanas, apósitos, piel sana) y resaltar exclusivamente los hallazgos patológicos.
- **Clasificadores Independientes (Trainable):** Las capas densas finales se entrenan para mapear la representación global de la bolsa hacia los logits de clasificación MIML, permitiendo diagnósticos multietiqueta precisos.

5.3 Mitigación de Sesgo Tisular

Dada la prevalencia dispar entre tejidos (ej. alta frecuencia de granulación frente a la rareza del tejido necrótico), se implementó el cálculo dinámico de `pos_weight` (W_c). Este factor amplifica el castigo en la función de pérdida `BCEWithLogitsLoss` cuando el modelo falla en detectar clases minoritarias, garantizando una sensibilidad clínica equilibrada en todas las ramas de atención.

6 Fase 2: Evaluación y Métricas MIML

6.1 Cuantificación del Error mediante Hamming Loss

Dada la naturaleza multietiqueta del diagnóstico de úlceras, la métrica de exactitud simple resulta insuficiente al ignorar aciertos parciales. Se adoptó la pérdida de Hamming (L_H) como criterio primario de validación [?]:

$$L_H = \frac{1}{|D|} \sum_{i=1}^{|D|} \frac{1}{K} |Y_i \Delta Z_i| \quad (3)$$

Donde Δ representa la diferencia simétrica (XOR) entre el vector real Y y la predicción Z para las K clases. Esta métrica permite penalizar de forma granular cada etiqueta individual, reflejando con mayor fidelidad la precisión del modelo en la detección de tejidos coexistentes.

6.2 Generación de Class Activation Maps (CAMs)

La arquitectura de K -ramas permite la extracción de mapas de atención independientes para cada tejido. Tras el entrenamiento de la Fase 1, estos mapas dejan de representar

ruido aleatorio y comienzan a converger espacialmente sobre las estructuras biológicas de la herida.

Estos mapas constituyen el "Potencial Unario" fundamental para la Fase 4 del proyecto, donde se integrará el refinamiento mediante *DenseCRF* para la obtención de máscaras de segmentación densa y la cuantificación volumétrica en cm^2 mediante el marcador ArUco.

7 Evolución y Optimización de la Interfaz Clínica de Ingesta Co-MIL

Para llevar a cabo la Fase de Anotación Clínica, se requirió evolucionar el script original de generación de bolsas hacia una herramienta gráfica robusta (`generador_bolsas.py`). Dado que la anotación será realizada por un experto médico sin formación en programación, la interfaz debía resolver tres retos principales: ergonomía clínica, preservación rigurosa de la topología tensorial y trazabilidad de los datos en un entorno de trabajo colaborativo.

7.1 Flujo de Trabajo y Experiencia de Usuario (UX)

Se diseñó una arquitectura de interfaz asimétrica utilizando la biblioteca `customtkinter`, priorizando el espacio visual de la fotografía médica. El flujo de trabajo clínico consta de los siguientes pasos y controles integrados:

- **Carga y Navegación:** El botón *“Cargar Carpeta Dataset”* inicializa el lote de trabajo. La navegación se realiza mediante los botones *“.Anterior”* y *“.Siguiente”*, además de un control de salto automático a la próxima imagen pendiente.
- **Macrolocalización Multirregión:** El médico delimita interactivamente una o varias Regiones de Interés (ROIs) trazando cajas delimitadoras (Bounding Boxes). El sistema gestiona múltiples ROIs independientes por fotografía mediante el botón *“Guardar ROI Local”*.
- **Anotación Débil y Catálogo Dinámico:** Mediante un panel lateral, se asignan las etiquetas presentes (ej. Fibrina, Tejido Necrótico, Exudado). Se implementó un control de *“Renombrar etiquetas del catálogo”* que abre una ventana modal independiente, permitiendo ajustar la nomenclatura base en vivo y sincronizando los cambios en un archivo de configuración maestro (`class_catalog.json`).
- **Gestión de Sesiones (Autosave):** Para evitar la pérdida de información en jornadas clínicas extensas, se implementó un sistema de autoguardado silencioso. Cada trazo o selección genera un archivo temporal `.json`; si la sesión se interrumpe y el usuario presiona *“SALIR & CERRAR APLICACIÓN”*, los recuadros no procesados se restauran automáticamente al reabrir la imagen.

7.2 Módulo de Auditoría Tensorial (Modo Investigador)

Para fines de depuración y análisis algorítmico, se integró un visor de parches avanzado que se activa con el botón *“Visión de Parches (Inspector MIL)”*. Esta herramienta modal despliega la cuadrícula exacta con la que el modelo fragmentará la herida.

El inspector cuenta con un control de *zoom* dinámico interpolado mediante filtros *LANCZOS*, un botón para exportar la malla visual y un panel de análisis que detalla: la firma del anotador, las dimensiones reales del Bounding Box expandido, el número total de instancias N (parches) y la validación del vector Y global para esa ROI específica.

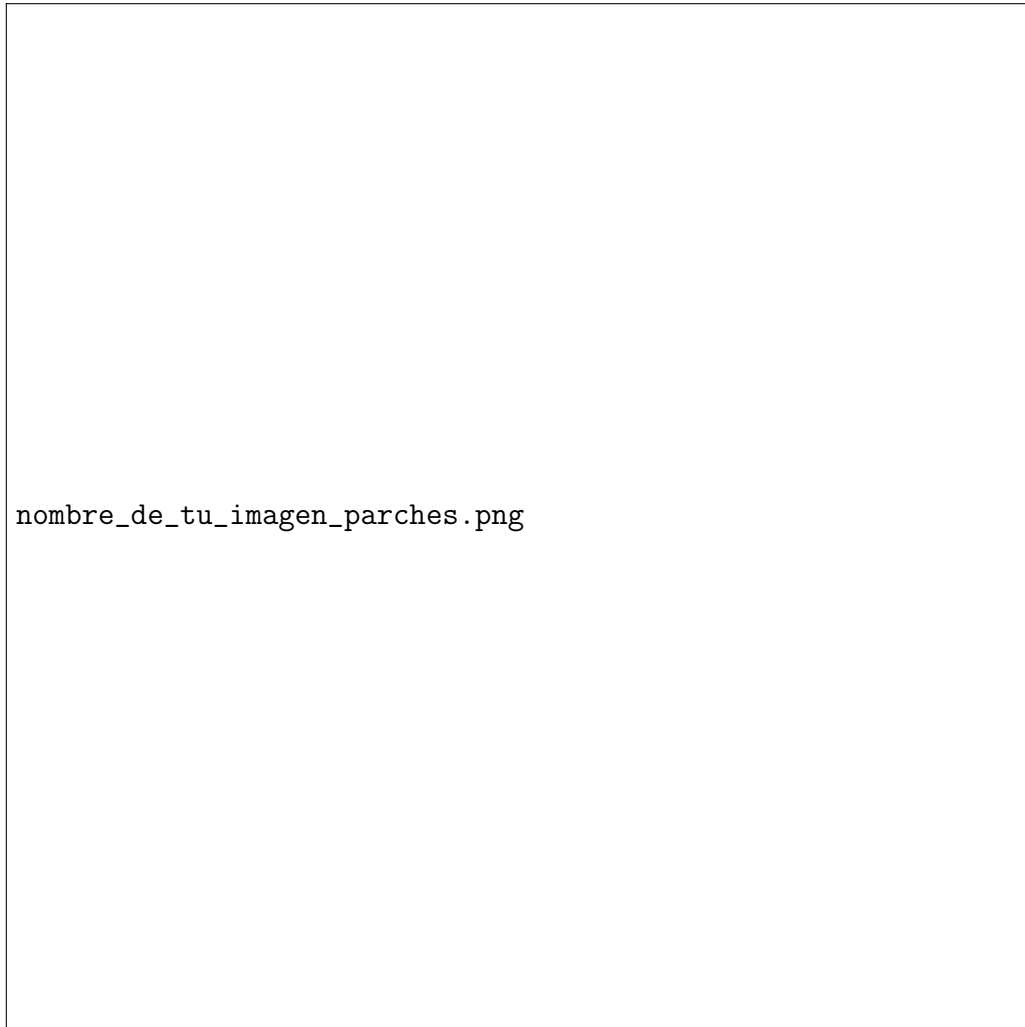


Figura 1: Captura de pantalla del "Modo Investigador" mostrando la malla superpuesta en color cyan, los índices de las instancias ($P - 00$, $P - 01 \dots$) y el panel inferior de análisis estadístico de la bolsa MIL.

7.3 Conservación del Core Matemático

Para garantizar que la arquitectura *MobileNetV2* no reciba gradientes falsos en los bordes de los tensores, la extracción de la bolsa (*Unfold*) realiza un preprocesamiento invisible para el médico:

1. **Expansión Inteligente (Smart Crop):** El recuadro dibujado se expande matemáticamente absorbiendo tejido real hasta que la altura y anchura sean múltiplos exactos del tamaño de parche (224×224 px).
2. **Reflection Padding:** Cuando la expansión geométrica supera los límites físicos de la fotografía original, se emplea la función `torch.nn.ReflectionPad2d`. Este

operador genera un efecto espejo que copia la textura perilesional, manteniendo la derivada continua espacial y evitando el uso de *Zero-Padding* (relleno negro).



nombre_de_tu_imagen_reflection.png

Figura 2: Ejemplo comparativo visual de la extracción de instancias en los límites de la fotografía, mostrando la preservación topológica tras la aplicación de *Reflection Padding*.



Figura 3: Diagrama de flujo de la lógica de interacciones de la interfaz: desde la carga de la imagen de alta resolución hasta la serialización matemática de la bolsa y sus metadatos en un archivo `.pt` binario.